

Analysis: Tree Balancing

Analysis: Tree Balancing - AVL vs Red-Black Trees

1. Dataset Structure Inference

I inferred each dataset structure from the raw value order and then validated that inference against measured balancing behavior.

- **Dataset A:** Alternating low/high pattern with local zigzags, not globally sorted, with a few duplicates.
 - Observable pattern: values repeatedly jump between lower and higher ranges (for example 42, 901, 55, 887, ...).
 - Size/uniqueness: 305 values, 299 unique, 6 duplicates.
- **Dataset B:** Strictly ascending.
 - Every new key is greater than the previous key.
 - Size/uniqueness: 279 values, all unique.
- **Dataset C:** Strictly ascending with more elements than B and tighter numeric range.
 - Same structural stress pattern as B (right-leaning insertion path before balancing).
 - Size/uniqueness: 416 values, all unique.
- **Dataset D:** Strictly descending.
 - Mirror image of B (left-leaning insertion path before balancing).
 - Size/uniqueness: 229 values, all unique.
- **Dataset E:** Pseudo-random mixed order with many duplicates.
 - No monotonic order; repeated values are frequent.
 - Size/uniqueness: 309 values, 268 unique, 41 duplicates.

This matters because both implementations route duplicates to the right subtree (else branch), which adds directional pressure in duplicate-heavy datasets.

2. Instrumentation and What Was Measured

Both implementations were instrumented and executed per dataset.

AVL (AVLTree.cpp)

Tracked metrics: - rotations - balance_checks - final_height - max_height - height_history (height after each insertion)

Code path tied to metrics: - In `insert_iterative`, every ancestor visited during upward rebalance increments `balance_checks` once. - Rotations are counted in `rotate_left` and `rotate_right`. - Height after each insertion is appended to `height_history_`.

Red-Black (RBTree.cpp)

Tracked metrics: - rotations - recolors (strict semantics: only when a node color actually changes) - final_height - max_height - height_history (height after each insertion)

Code path tied to metrics: - Rotations are counted in `rotate_left` and `rotate_right`. - Recolors are counted through `set_color(node, color)` only when `node->color != color`. - `fix_violation` drives recolor/rotation activity (uncle-red recolor cascade vs uncle-black rotation cases). - Height after each insertion is appended to `height_history_`.

Artifacts: - Aggregate table: `results_table.md` / `results_table.pdf`

3. Results Table

Dataset	AVL rot	AVL bal chk	AVL fin H	AVL max H	RB rot	RB recolors	RB fin H	RB max H
DatasetA	230	2211	10	10	185	755	12	12
DatasetB	270	2000	9	9	265	1323	14	14
DatasetC	407	3233	9	9	401	2007	15	15
DatasetD	221	1577	8	8	216	1075	13	13
DatasetE	172	2119	10	10	138	653	10	10

rot = rotations, bal chk = balance_checks, fin H = final height, max H = max height

4. How Each Dataset Stresses Each Tree

Dataset A (alternating low/high with minor duplication)

- **AVL stress:** High rotations (230) and high checks (2211). The alternating pattern produces repeated local left-right and right-left shape pressure; AVL corrects aggressively due to strict balance factor bounds.
- **RB stress:** Substantial recolor activity (755) and moderate rotations (185). Alternating inserts produce frequent red-red violations in varying subtrees, causing recolor cascades and occasional rotations.
- **Difference:** AVL keeps height lower (10 vs 12) at the cost of more rotations/checking.

Dataset B (ascending)

- **AVL stress:** 270 rotations and 2000 checks from repeated right-side growth. Every insertion tends to extend the same side before rebalancing.
- **RB stress:** 1323 recolors and 265 rotations. The same directional growth causes repeated red-red repairs; many fixes are recolor-heavy with periodic structural rotations.
- **Difference:** RB allows much taller shape (14 vs AVL 9), showing its looser balance philosophy.

Dataset C (longer ascending)

- **AVL stress:** Highest AVL workload: 407 rotations and 3233 checks. Same stress type as B but amplified by sequence length.
- **RB stress:** Highest RB workload too: 2007 recolors and 401 rotations. Long monotonic growth repeatedly triggers violation repair.
- **Difference:** Biggest height gap appears here (RB 15 vs AVL 9), strongest evidence that AVL trades more strict balancing work for lower height.

Dataset D (descending)

- **AVL stress:** Mirror of B with left-side pressure: 221 rotations, 1577 checks.
- **RB stress:** Mirror-like behavior as well: 1075 recolors, 216 rotations.
- **Difference:** Same pattern as ascending, just mirrored: AVL remains significantly shorter (8 vs 13).

Dataset E (random with many duplicates)

- **AVL stress:** Lowest AVL rotations (172) but still many checks (2119) due to ancestor traversal on each insert.
- **RB stress:** Lowest RB rotations (138) and recolors (653) among datasets; random order reduces long repair cascades.
- **Difference:** Both end at the same final height (10). This is the least adversarial pattern for both trees in this assignment.

5. Why They Respond Differently (Code-Driven Explanation)

- AVL implementation checks balance on each ancestor after every insert. This guarantees tight height control but creates persistent balancing overhead (`balance_checks` + rotations), especially for monotonic patterns.
- RB implementation does not compute balance factor and does not force immediate strict height parity. It only repairs rule violations when parent is red. This shifts work from “height correction everywhere” to “color-rule repair where needed.”
- In monotonic datasets, RB often performs many recolors because uncle-red cases propagate upward; in AVL, the same pattern causes many rotations and checks.
- With strict recolor counting, RB recolor totals dropped from assignment-style assignment-count values to true color-change counts, but the relative trend remained the same ($C > B > D > A > E$).

6. Efficiency per Dataset and Robustness Overall

Per-dataset efficiency

- If efficiency is defined as **lower final height**, AVL wins on A/B/C/D and ties on E.
- If efficiency is defined as **fewer rotations**, RB wins on all five datasets.
- If efficiency is defined as **total corrective actions in its own policy space**, monotonic datasets are expensive for both, random is least expensive for both.

Robustness across all datasets

- AVL height range: **8 to 10**.
- RB height range: **10 to 15**.

For this assignment’s focus on balancing behavior under varying input structures, AVL is more robust in maintaining consistently low height across all patterns. RB is robust in a different sense (fewer rotations), but it tolerates larger height growth on structurally adversarial orders.

7. Reflection

- Dataset structure directly controls which balancing mechanism gets stressed: AVL height constraints vs RB color constraints.
- Monotonic insertion orders are the clearest stressors for both trees, but symptoms differ: AVL rotation/check pressure, RB recolor cascade pressure.
- Instrumentation converts abstract balancing rules into measurable evidence. The per-insert height history files were especially useful for confirming not only final shape but growth trajectory during insertion.